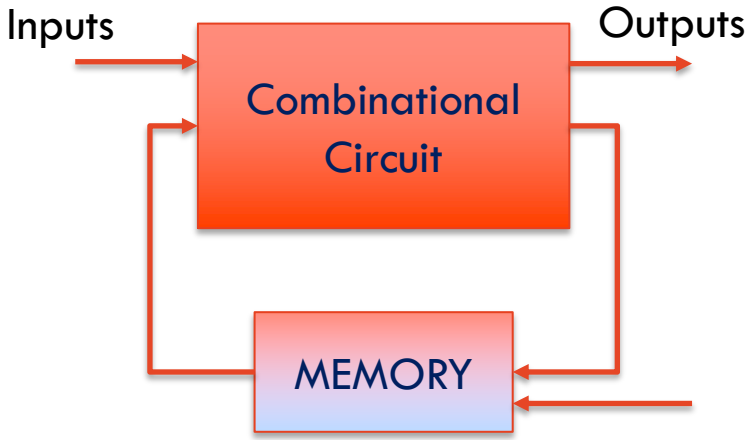
Sequential Design
Finite State Machines

Viorela Ila

Sequential Design - Recap

- In **Combinational logic,** outputs depend only on current inputs.
- In **Sequential logic** outputs depend on both **current inputs** and the **history of inputs,** usually maintained by clocked elements like flip-flops.
- **Sequential design** in Verilog typically involves the use of flip-flops or registers to store the state of the system and control the flow of data.





Combinational Logic vs. Sequential Logic

```
// 4-to-1 Multiplexer (Combinational)
module mux4to1 (
    input wire [1:0] a, b, c, d,
    input wire [1:0] sel,
    output reg [1:0] y );
    always @(*) begin // Combinational always
block
        case (sel)
            2'b00: y = a;
            2'b01: y = b;
            2'b10: y = c;
            2'b11: y = d;
            default: y = 2'b00;
        endcase
    end
endmodule
```

```
// 3-bit Up Counter (Sequential)
module counter3bit (
    input wire clk, reset, enable,
    output reg [2:0] count );
    always @(posedge clk) begin
        if (reset)
            count <= 3'b000;
        else if (enable)
            count <= count + 1; // If enable
is low, count holds its value
    end
endmodule
```

Sequential Design - Recap

- **Clock Signal:** Sequential circuits rely on a clock signal to synchronize changes in the state. The posedge keyword indicates that the logic should update on the rising edge of the clock.
- **Registers and Flip-Flops:** These are the basic storage elements in sequential circuits, used to hold the state or data across clock cycles.
- **Reset Signal:** A reset signal is often used to initialize the state of the circuit to a known value, typically at startup or in response to a specific event.
- **Timing Considerations:** Sequential circuits must be designed with timing constraints in mind, including **setup** and **hold** times for flip-flops and propagation delays.
- **Synchronous vs. Asynchronous**: Synchronous sequential designs rely on a clock signal, while asynchronous designs do not.

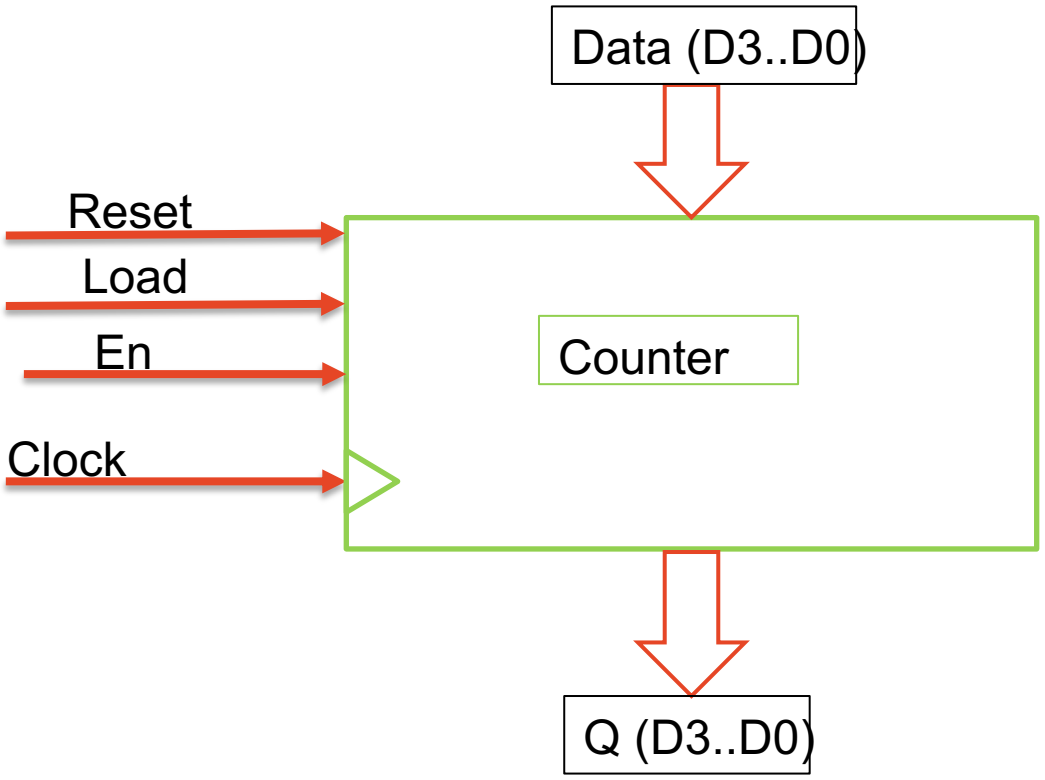
Counter – Variant 1

```
module counter (
    input clk,            // Clock input
    input reset,          // Synchronous reset input
    output reg [3:0] q    // 4-bit output
);

    // Always block for sequential logic
    always @(posedge clk) begin
        if (reset) // synchronous reset
            q <= 4'b0000; // On reset, set counter to 0
        else
            q <= q + 1;   // Otherwise, increment the counter
    end

endmodule
```

Counter – Variant 2




```
// Binary Counter
//
module Counter (
  input wire [3:0] d,
  input wire clk, reset, load, en,
  output reg [3:0] q
  );
  always @(posedge clk)
  begin
    if (reset)  // synchronous reset
      q <= 0;
    else if (load == 1'b1)
      q <= d;
    else if (en == 1'b1)
      q <= q + 1;
  end
endmodule
```

Counter – Variant 2 RTL Viewer

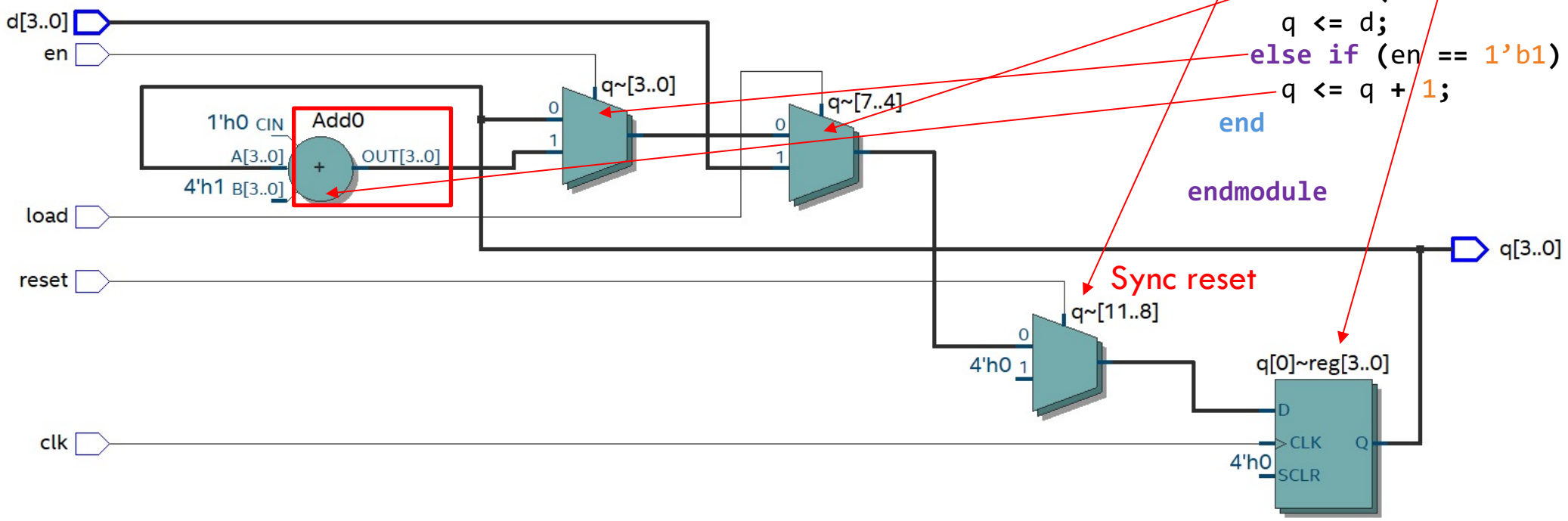
Quartus synthesizes an adder for the `q <= q + 1;`

See the RTL netlist diagram below and how the if-else statements become multiplexers:

```
// Binary Counter
//
module Counter (
   input wire [3:0] d,
   input wire clk, reset, load, en,
   output reg [3:0] q
  );

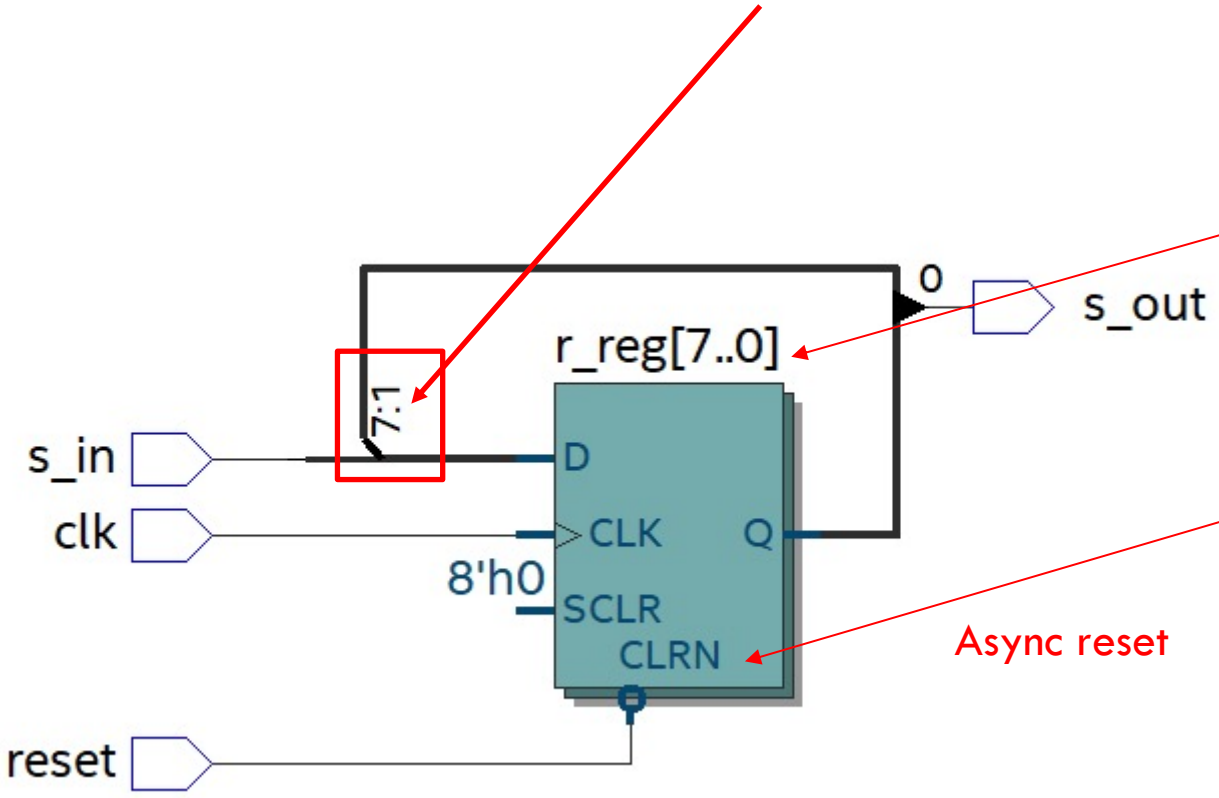
  always @(posedge clk)
  begin
    if (reset)  // synchronous reset
       q <= 0;
    else if (load == 1'b1)
       q <= d;
    else if (en == 1'b1)
       q <= q + 1;
  end

endmodule
```



Shift Register

Quartus synthesizes a collection of D flip-flops.
Note how `D[7:0] = {s_in, Q[7:1]}` in the RTL netlist diagram below:




```
// Example of shift register
module free_run_shift_reg
    #(parameter N=8)
    (
     input wire clk, reset,
     input wire s_in,
     output wire s_out
    );

    reg [N-1:0] r_reg;
    wire [N-1:0] r_next;

    always @(posedge clk, negedge reset)
    begin
       if (~reset)
          r_reg <= 0;
       else
          r_reg <= r_next;
    end

    assign r_next = {s_in, r_reg[N-1:1]};
    assign s_out = r_reg[0];

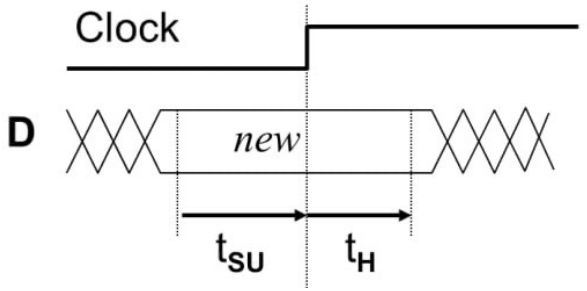
endmodule
```

Timing Analysis in Sequential Design

Timing Parameters in Sequential Circuits:

Clock Period (T): The time interval between two consecutive rising or falling edges of the clock.

$T = \frac{1}{f_t}$ where f_t is the clock frequency.



Setup Time (T_{SU}): The minimum time before the clock edge that the data input must be stable.

Hold Time (T_H): The minimum time after the clock edge that the data input must remain stable.

Clock-to-Q Delay (T_{CQ}): The time taken for the output (Q) to change after the clock edge.

Propagation Delay (T_{PD}): The time taken for a signal to propagate through combinational logic between flip-flops.

Timing Analysis in Sequential Design

Critical Path and Maximum Frequency

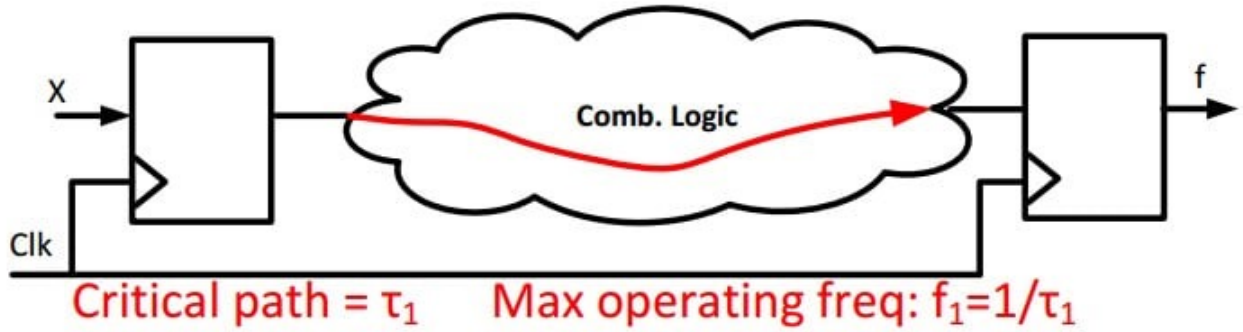
- **In most situations there are many paths in the circuit.**

- **Critical Path:** The longest path in the sequential circuit that determines the clock period.
  - Includes the combinational logic delay (T_{PD}) and clock-to-Q delay (T_{CQ}).

- **Maximum Operating Frequency (f_{max}):**

$$f_{\max} = \frac{1}{T_{clk}}$$

$$T_{clk} = T_{SU} + T_{CQ} + T_{PD} + T_H$$



  - Ensures that the design meets timing requirements.

FPGA Design and Simulation - Recap

An HDL Simulator is just a Software Program

- It models the actions of hardware, but also includes programmatic constructs and interactive elements.
- It is based on TCL scripting, it is a complete and powerful development environment.
- It uses clever programming techniques to create the appearance of **concurrency** to correctly model how real hardware behaves.
- Some HDL code will simulate but not synthesize. Some HDL code will synthesize but not simulate. Your HDL code describing hardware should both simulate and be synthesizable.

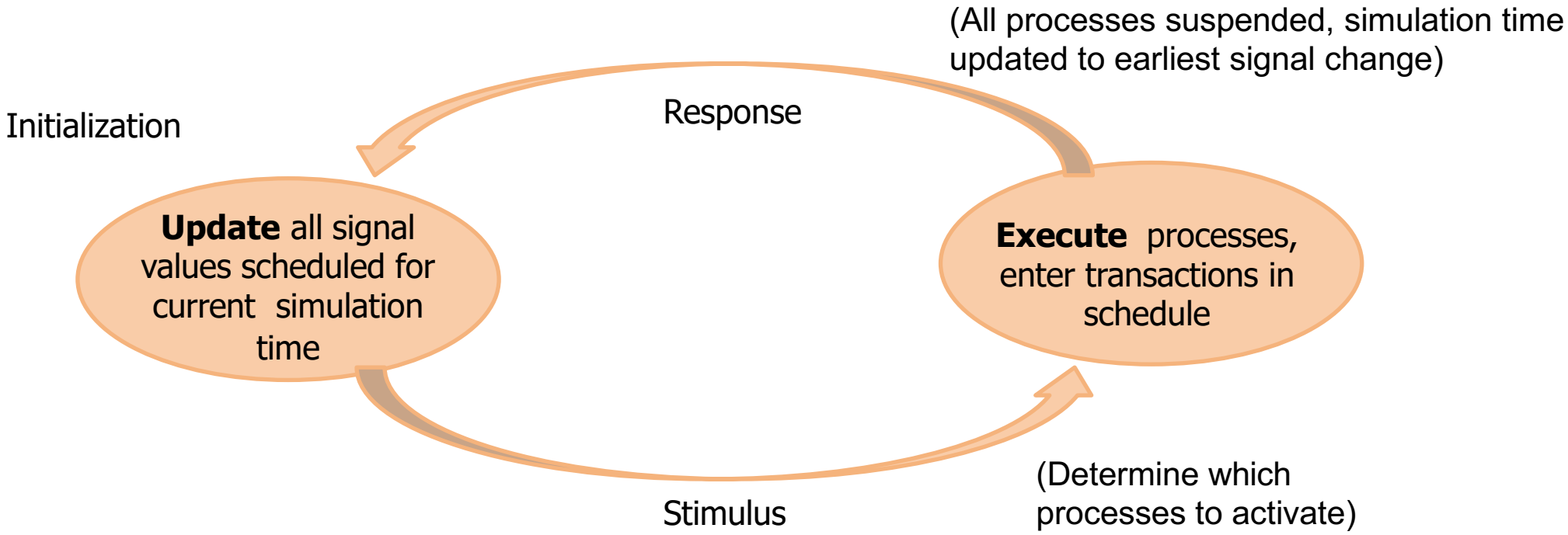
FPGA Design and Simulation - Recap

Main simulation concepts:

- **Simulation Cycle –** Stimulus and Response
- **Simulation Timing**
- **Sensitivity Lists:** create events that happen that have to be timed in simulation.
- **Signal Drivers:** : A driver is a concurrent process that determines the value of a signal. Signal drivers are those which are feeding some value to output line. If in a program you connect multiple types of these drivers to a single output in different procedural blocks or assignment statements then you get an error.
- **Resolution Functions:** the mechanism the simulator uses to determine the final value of a net (wire) when multiple drivers are connected to it simultaneously.

Stimulus/Response Paradigm

- Stimulus (inputs) changes at specific **simulation time**
- Processes evaluate resulting logic changes
- Signal values change after specific simulation time has elapsed
- All concurrent processes repeatedly execute in parallel (concurrently), following a standard simulation cycle



Delta Cycles

- In an event driven logic simulator, the concept of **physical time** is abstracted away. The simulator only cares about changes on signals.

- Assuming that the RTL has no delays in its description and the only delays in your testbench are in the generation of your clock, the simulation executes exactly the same regardless of what the clock period is.

- **Delta cycles** come into play within that **instantaneous moment when everything gets evaluated**. It is basically a **simulation time step without advancing physical time**. Whenever a signal changes, that signal output fans out to the input of other logic that may cause other signals to change.

- Delta cycles help us understand how a simulator represents concurrent, zero-delay hardware behaviour. They matter when interpreting signal-update order, debugging simulations, and understanding why some HDL code may not settle

Delta Cycles

Simulation time advances in discrete **time steps** (e.g., 10 ns)

```
assign b = a;
assign c = b;
```

Simulation time = 10 ns

Delta 0: a changes
↓
Delta 1: b updates
↓
Delta 2: c updates

Simulation time is still 10 ns

Finite State Machines



Traffic Lights:

- States: Green, Yellow, Red
- Transitions: Timed changes or triggered by sensors



•Elevator Control:

- States: Idle, Moving Up, Moving Down, Door Open, Door Closed
- Inputs: Button presses, floor sensors



•Vending Machine:

- States: Idle, Waiting for Payment, Dispensing
- Inputs: Coins inserted, selection pressed

Finite State Machines

I - **Input States:** Set of possible value combinations of input signals

O - **Output States:** Set of possible value combinations of output signals

S - **Internal States:** Set of possible value combinations of the internally stored signals

Two functions that define the working of the circuits

$f : S \times I \rightarrow S$ – **Next state function:** it associates an internal state to every pair (internal state, input state)

$h : S \times I \rightarrow O$ – **Output function:** it associates an output state to every pair (internal state, input state)

Finite State Machines – Traffic Light



I = {0, 1} (timer expired 0 = no, 1 = yes) - Input States

O = {001, 010, 100} (Green, Yellow, Red)- Output States

S = {G, Y, R} - Internal States

f(s, 0) = s, ∀ s ∈ S – Stay in the current state

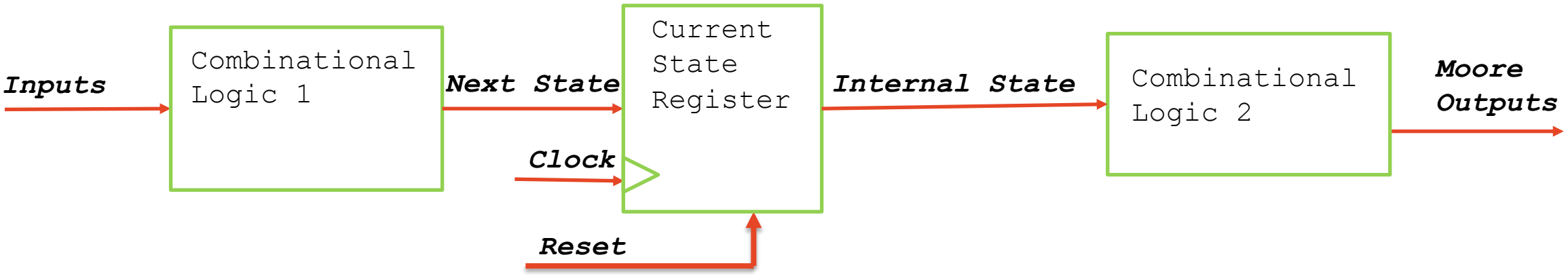
f(G, 1) = Y, f(Y, 1) = R, f(R, 1) = G – Next state function

h(G) = 001, h(Y) = 010, h(R) = 100 – Output function

Finite State Machines

In **Moore** machines the output only depends on the internal state.

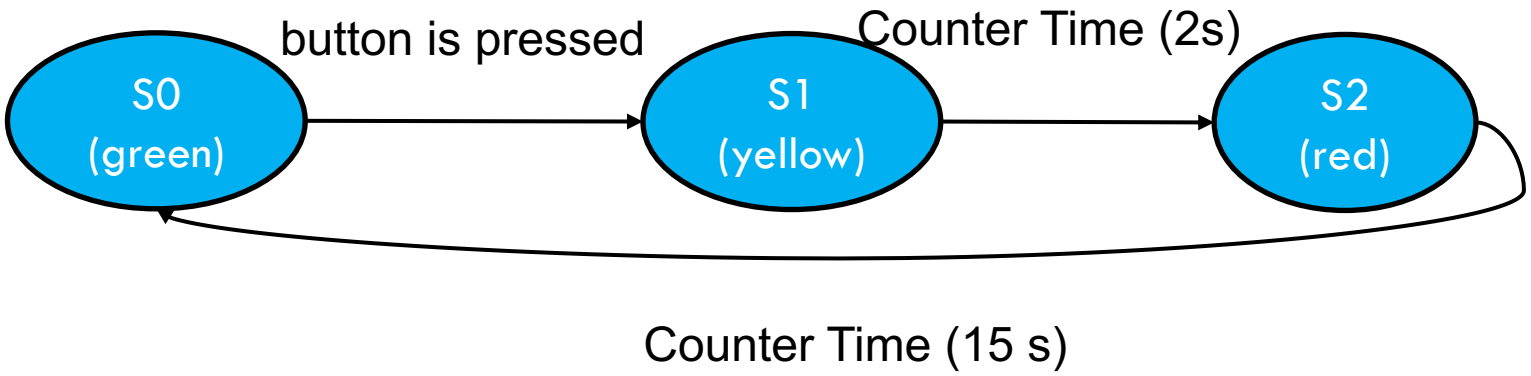
h : S ⟶ O – Output function only depends on the current state



Traffic Light with Pedestrian Input

- **States:** Green, Yellow, Red
- **Inputs:** A button pressed by a pedestrian requesting to cross.
- **Outputs:** The light colour — Green, Yellow, or Red.

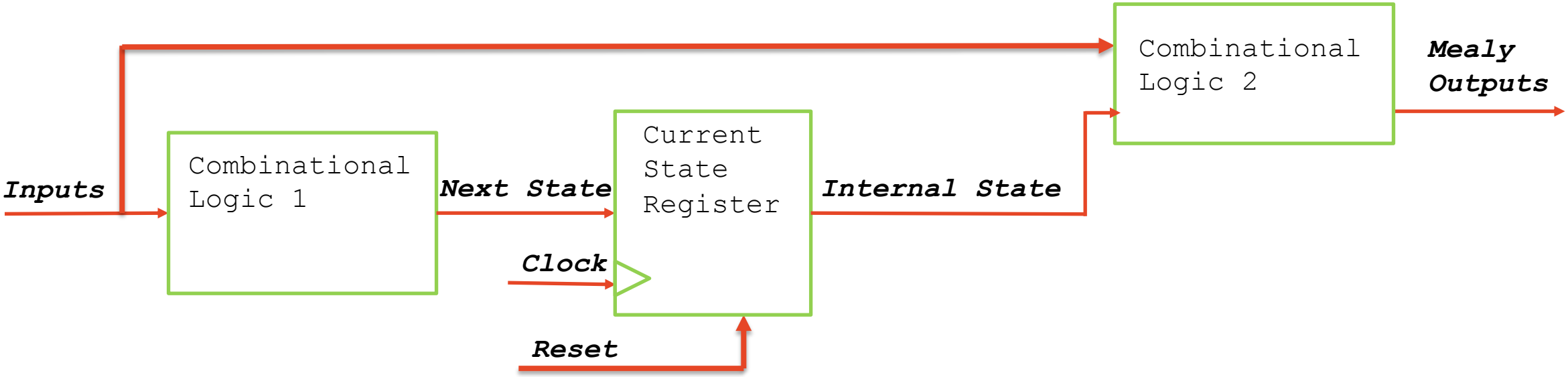
State	**Output** (Light)	**Description**
S0	Green	Cars move, pedestrians wait.
S1	Yellow	Warning, about to turn red.
S2	Red	Cars stop, pedestrians cross.



Finite State Machines

In **Mealy** machines the inputs and the state drive the output.

h : S x I ⟶ O – Output function depends on the input and internal states

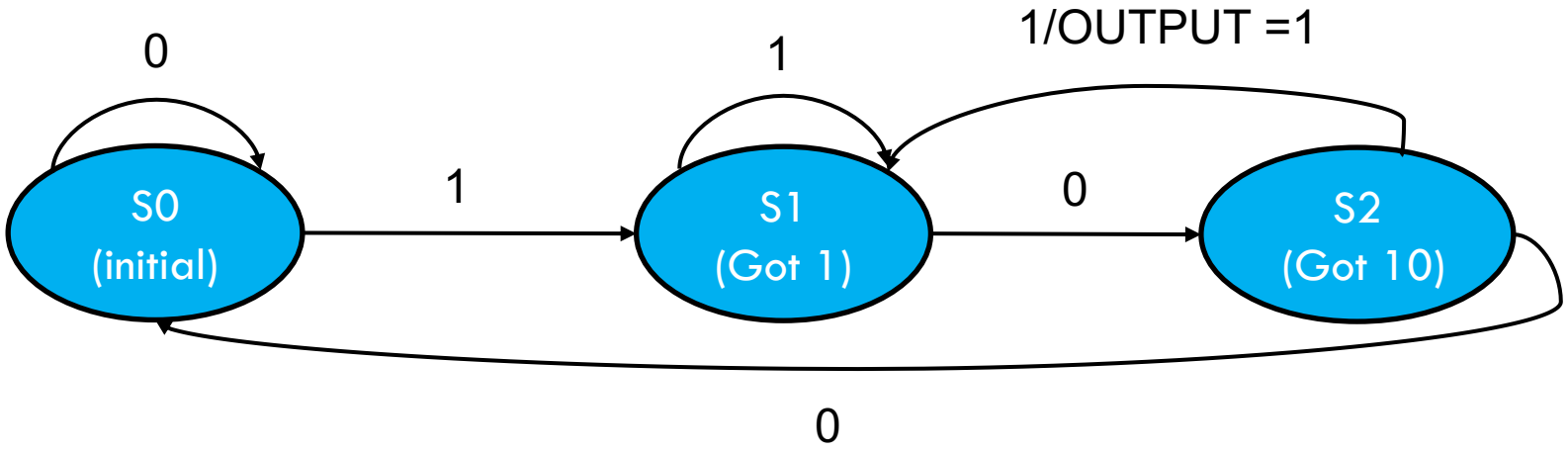


Sequence Detector – Detect "101"

FSM detects the bit pattern **"101"** in a serial input stream.
Whenever the pattern is detected, it outputs 1 for **one clock cycle**; otherwise, it outputs 0.

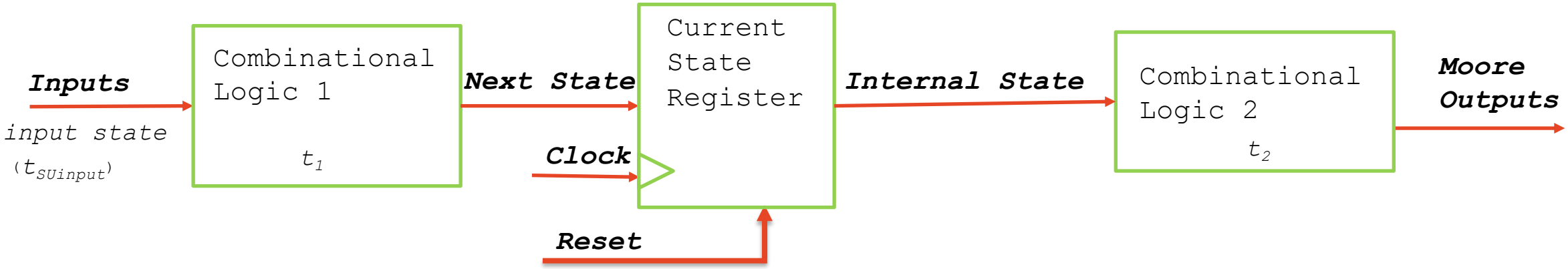
Input: X → Serial bit stream (0 or 1)
Output: Z → Pattern found flag (1 when 101 is detected)
States:
- **S0:** Idle (no match yet)
- **S1:** Matched first 1
- **S2:** Matched 10

```
Input: 1 0 1 1 0 1 0 1 1 0 1 0 1
State: S0→S1→S2→S1→S1→S2→S1→S0→S1→S1→S2→S1→S0
Output: 0 0 1 0 0 1 0 0 0 0 1 0 0
```

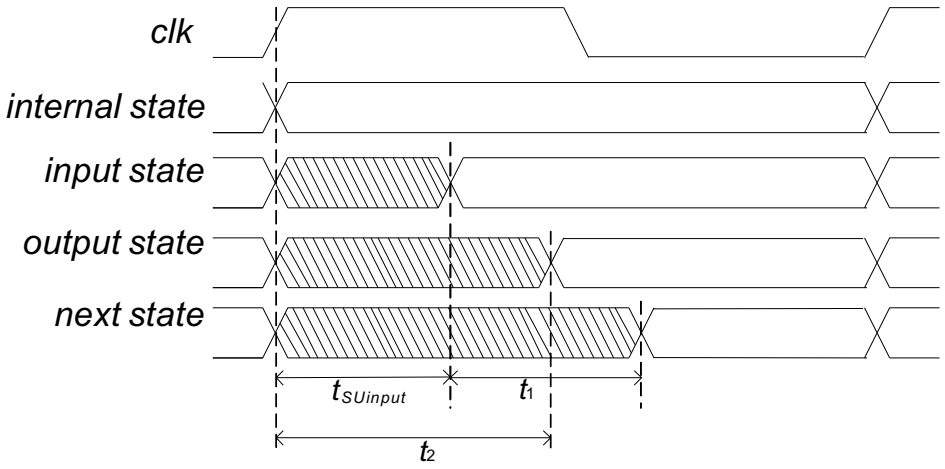


Finite State Machines - Delays

h : S → O – Output function only depends on the current internal state

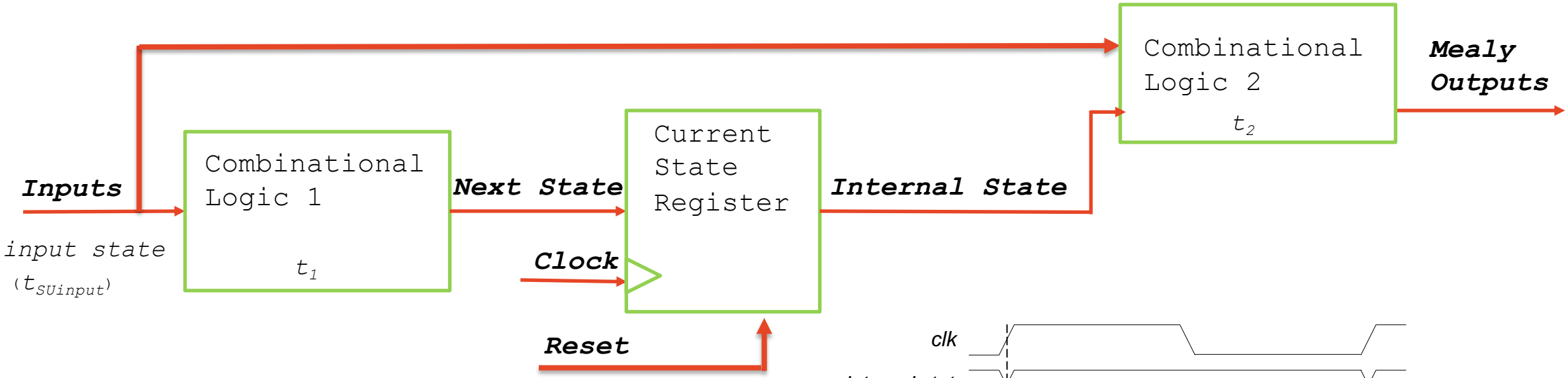


$T_{CLK} > max \{ t_{SUinput} + t_1 , t_2 \}$

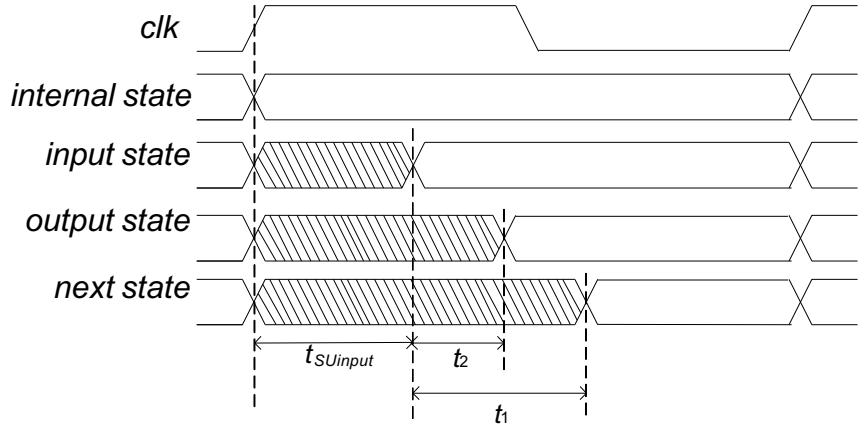


Finite State Machines - Delays

h : S x I ⟶ O – Output function depends on the input and internal states



$T_{CLK} > max\ \{\ t_{SUinput} + t_1\ ,\ t_{SUinput} + t_2\}$



FSM to Control a Robot

Implement an obstacle avoidance behaviours for a robot.

- The robot is equipped with Left and Right Bumpers that can detect objects.
- We want the robot to **back up** when it senses an object then **turn right** if the object is sensed by the **left** bumper or **turn left** if the **right** bumper senses an object.
- When the robot hasn't hit anything it will just drive **forward**.

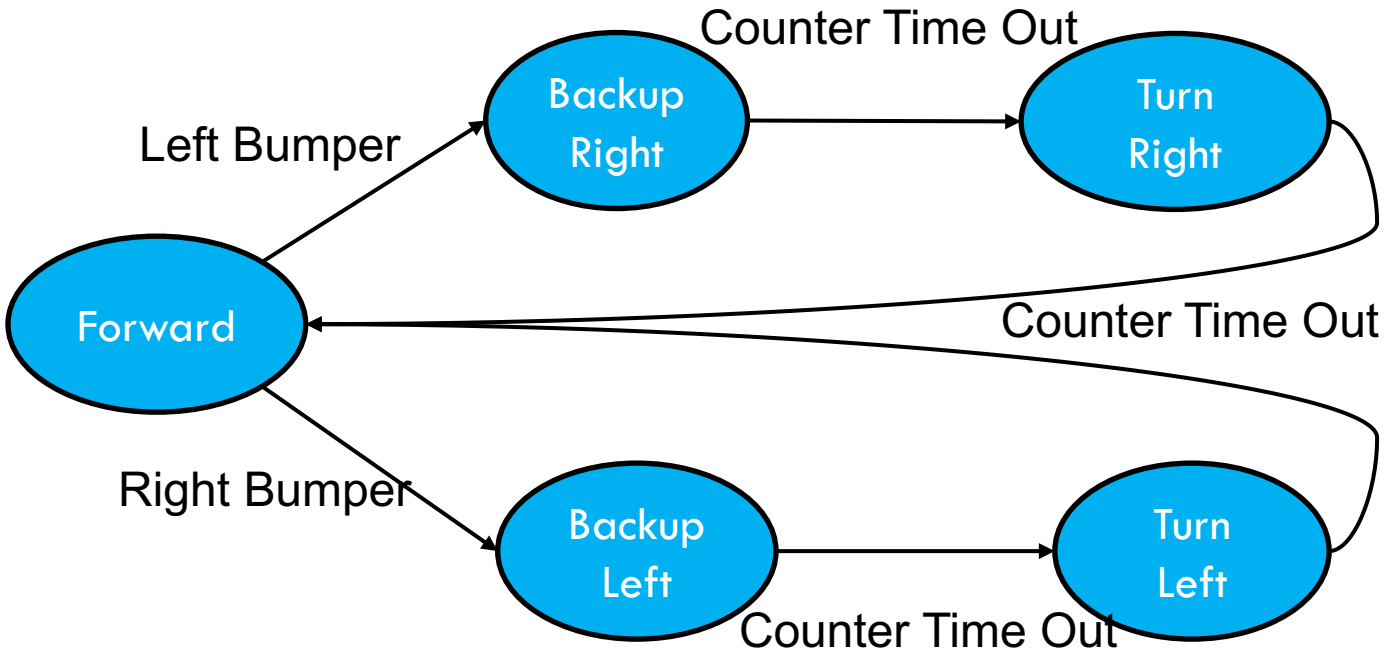
This leads us to five states:

- **FORWARD**
- **BACKUP_RIGHT**
- **TURN_RIGHT**
- **BACKUP_LEFT**
- **TURN_LEFT**



There are two versions of the **BACKUP** state, **BACKUP_RIGHT** and **BACKUP_LEFT**. This is because we need to encode in the states which one will turn right after it backs up and which will turn left. If we didn't have separate states, we would have to read the state of the bumper after it backed up and at that point the bumper would not be pressed anymore.

FSM to Control a Robot



State Transitions:

When the FSM is in the **FORWARD** state it should change to the **BACKUP_RIGHT** state when the ***left bumper*** is pressed, or to the **BACKUP_LEFT** state when the ***right bumper*** is pressed.

When FSM is in **BACKUP_RIGHT** and ***Counter Time Out*** the state changes to **TURN_RIGHT.**

When FSM is in **BACKUP_LEFT** and ***Counter Time Out*** the state changes to **TURN_LEFT.**

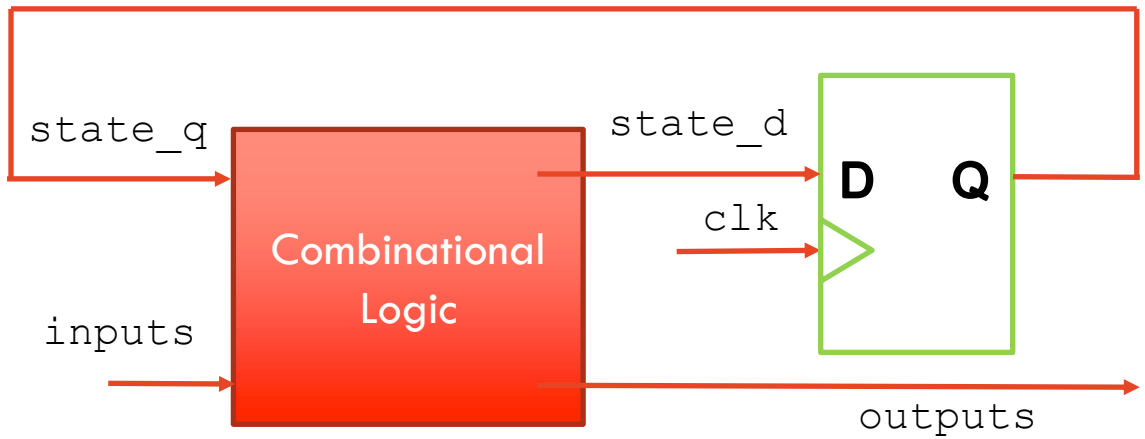
When FSM is in either **TURN_RIGHT** or **TURN_LEFT** and ***Counter Time Out*** the state changes to **FORWARD**

FSM to Control a Robot

```
module robot_fsm (
    input clk,
    input rst,
    input bumper_left, //signals from the bumper
    input bumper_right, //signals from the bumper
    output reg [7:0] left, //signals to the motors
    output reg [7:0] right //signals to the motors
  );

  localparam STATE_SIZE = 3;

  localparam FORWARD = 0,
    BACKUP_RIGHT = 1,
    TURN_RIGHT = 2,
    BACKUP_LEFT = 3,
    TURN_LEFT = 4;
```



```
reg [STATE_SIZE-1:0] state_d, state_q;
reg [25:0] ctr_d, ctr_q;

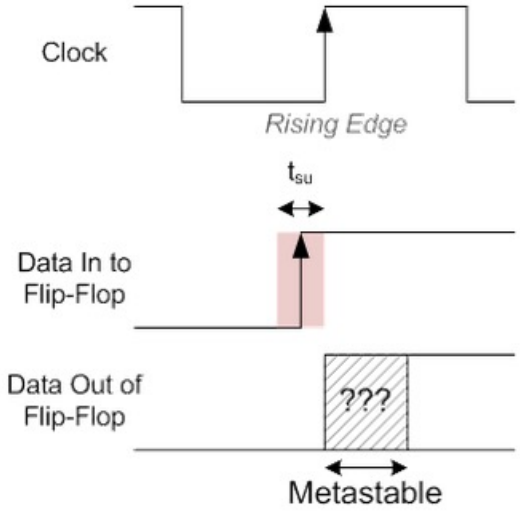
reg [1:0] sl_d, sl_q, sr_d, sr_q;

always @(*) begin
  sl_d[0] = bumper_left;
  sl_d[1] = sl_q[0];
  sr_d[0] = bumper_right;
  sr_d[1] = sr_q[0];

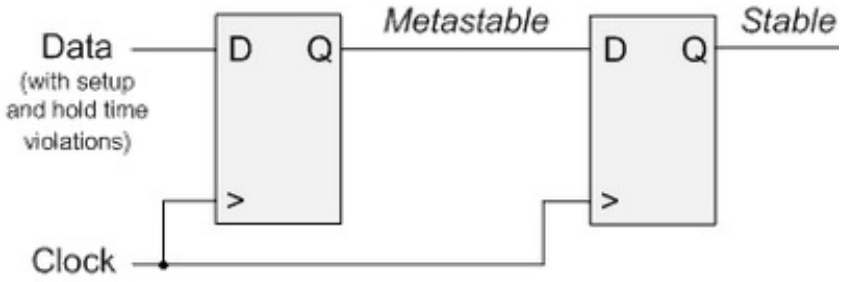
  ctr_d = ctr_q + 1'b1;
  state_d = state_q;
```

How to handle asynchronous events

A **metastable** state is one in which the output of a Flip-Flop inside of your FPGA is unknown, or non-deterministic.



"Double-flop" your data!!



```
case (state_q)
  FORWARD: begin
    ctr_d = 25'd0;
    if (sr_q[1]) begin
      state_d = BACKUP_LEFT;
    end else if (sl_q[1]) begin
      state_d = BACKUP_RIGHT;
    end
  end
  BACKUP_RIGHT: begin
    if (ctr_q == {26{1'b1}})
      state_d = TURN_RIGHT;
  end
  TURN_RIGHT: begin
    if (ctr_q == {25{1'b1}})
      state_d = FORWARD;
  end
  BACKUP_LEFT: begin
    if (ctr_q == {26{1'b1}})
      state_d = TURN_LEFT;
  end
  TURN_LEFT: begin
    if (ctr_q == {25{1'b1}})
      state_d = FORWARD;
  end
  default: state_d = FORWARD;
endcase
```

Body of FSM is case statement. **Next State Logic.**

When the counter is found to be high enough, the state transitions from **BACKUP_XXX** to **TURN_XXX**. Once it gets high enough again, it transitions from **TURN_XXX** to **FORWARD**.

The counter will overflow during the first transition because it reached the top, and when the FSM reaches the **FORWARD** state the counter is held at 0.

```
  case (state_q)
    FORWARD: begin
      left = 8'h50;
      right = 8'h50;
    end
    BACKUP_RIGHT: begin
      left = 8'hB0;
      right = 8'hB0;
    end
    TURN_RIGHT: begin
      left = 8'hB0;
      right = 8'h50;
    end
    BACKUP_LEFT: begin
      left = 8'hB0;
      right = 8'hB0;
    end
    TURN_LEFT: begin
      left = 8'h50;
      right = 8'hB0;
    end
    default: begin
      left = 8'h80;
      right = 8'h80;
    end
  endcase
end
```

Output Logic: Combinational logic to determine the output of the FSM, output only depends on current state.

FSM to Control a Robot

```
  always @(posedge clk) begin
    if (rst) begin
      state_q <= FORWARD;
    end
    else begin
      state_q <= state_d;
    end
    ctr_q <= ctr_d;
    sl_q <= sl_d;
    sr_q <= sr_d;
  end

endmodule
```

The current state register is created by use of an always block with the rising edge of the clock, and reset the position is set to state FORWARD.

ctr_q → combinational logic → ctr_d→ flip-flops → ctr_q

FSM to Control a Robot

```
always @(posedge clk) begin
if (rst)
        ctr_q <= 26'd0;
else if (state_q == FORWARD)
         ctr_q <= 26’d0;
else
        ctr_q <= ctr_q + 1'b1;
end
endmodule
```

We could instead write a more conventional separate counter.

Then we don't need ctr_d at all.

State Encoding

No.	Binary	Gray	Johnson	One-Hot
0	000	000	0000	00000001
1	001	001	0001	00000010
2	010	011	0011	00000100
3	011	010	0111	00001000
4	100	110	1111	00010000
5	101	111	1110	00100000
6	110	101	1100	01000000
7	111	100	1000	10000000

FSM State Encoding

Quartus can automatically recognise FSMs in your Verilog code and change the encoding scheme used upon code analysis! This Quartus feature supports the following encodings:

- **Default**: Automatically chooses an encoding. Sequential for fewer than 5 states, one-hot for 5-49 states, and gray for 50 or more states.
- **One-Hot**: Uses N bits for N states. Each state is represented by a single 1 bit and the rest are 0.
  - E.g. 0001, 0010, 0100, 1000.
- **Sequential**: Uses binary encoding. States are encoded from 0 to N-1 in a binary format.
  - E.g. 0000, 0001, 0010, 0011, 0100, etc…
- **Gray**: Uses M bits where encodings for adjacent states differ by one bit. Can represent up to 2^M states.
  - E.g. 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, etc…
- **Johnson**: Also uses M bits and adjacent states differ by one bit. Can represent up to 2^M states but uses less logic than Gray encoding.
  - E.g. 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000
- **Compact**: Minimizes the number of bits used for encoding the states.
- **User**: Allows manual state encoding via the Verilog source. Modify state constant values to change encoding.

FPGAs: one-hot is often advantageous as it requires less next-state logic and is typically easier to route.

SystemVerilog

- A powerful extension of Verilog that blends the best of both design and verification into a single language.
- Some of the new synthesis features (the majority of new features are for verification, e.g. OOP testbenches):

  - **The logic type :** fixes the syntax problem in Verilog of **reg** vs **wire**.

```
logic [3:0] wire_and_reg_all_in_one;
```

  - **Enums and typedef :** like in C, can be used to create meaningful values for state variables.

```
typedef enum logic [1:0] {S0, S1, S2, S3} state_t;
```

  - **New always blocks :** explicitly tells the synthesizer the intention of the code block e.g. latches versus no latches.

```
always_ff @(posedge clk)
always_comb
always_latch
```

  - **Implicit port connection** : connects all ports that have the same name as variables in the instantiating module

```
simple_fsm DUT (.*);
```

FSM State Encoding

```
localparam STATE_SIZE = 3;

localparam FORWARD = 0,
  BACKUP_RIGHT = 1,
  TURN_RIGHT = 2,
  BACKUP_LEFT = 3,
  TURN_LEFT = 4;
```

- States are explicitly defined with specific binary values.
- You manually assign binary codes to each state.

- The binary encoding for each state is explicitly defined, so the synthesis tool uses exactly the values you specify.
- You have full control over the encoding strategy (e.g., binary, one-hot, Gray code).

```
typedef enum logic [2:0] {
    FORWARD,
    BACKUP_RIGHT,
    TURN_RIGHT,
    BACKUP_LEFT,
    TURN_LEFT
} state_t;
state_t state_d, state_q;
```

- States are defined as part of an enumerated type.
- Synthesis tool automatically assigns binary values. By default, it typically assigns them sequentially starting from 0, but this can depend on the tool and settings.

- You do not need to manually assign binary values (though you can override the default assignments
- You can control the encoding through synthesis tool directives, but this requires familiarity with your toolchain.

FSM to Control a Robot Verilog vs. System Verilog

```
module robot_fsm (
    input clk,
    input rst,
    input bumper_left, //signals from the bumper
    input bumper_right, //signals from the bumper
    output reg [7:0] left, //signals to the motors
    output reg [7:0] right //signals to the motors
  );

  localparam STATE_SIZE = 3;

  localparam FORWARD = 0,
    BACKUP_RIGHT = 1,
    TURN_RIGHT = 2,
    BACKUP_LEFT = 3,
    TURN_LEFT = 4;
```

```
module robot_fsm (
  input  logic       clk,
  input  logic       rst,
  input  logic       bumper_left,
  input  logic       bumper_right,
  output logic [7:0] left
  output logic [7:0] right);

  localparam COUNTER_MAX = 26'd50_000_00;

typedef enum logic [2:0] {
    FORWARD,
    BACKUP_RIGHT,
    TURN_RIGHT,
    BACKUP_LEFT,
    TURN_LEFT
  } state_t;
```

```
reg [STATE_SIZE-1:0] state_d, state_q;
reg [25:0] ctr_d, ctr_q;

reg [1:0] sl_d, sl_q, sr_d, sr_q;

always @(*) begin
  sl_d[0] = bumper_left;
  sl_d[1] = sl_q[0];
  sr_d[0] = bumper_right;
  sr_d[1] = sr_q[0];

  ctr_d = ctr_q + 1'b1;
  state_d = state_q;
```

```
state_t state_d, state_q;

  logic [25:0] ctr_d, ctr_q;
  logic [1:0]  sl_d, sl_q, sr_d, sr_q;

always_comb begin
    sl_d[0] = bumper_left;
    sl_d[1] = sl_q[0];
    sr_d[0] = bumper_right;
    sr_d[1] = sr_q[0];

     // Defaults
    state_d = state_q;
    ctr_d   = ctr_q + 1'b1;
    // Default motor commands (stop)
    left  = 8'd0;
    right = 8'd0;
```

```
case (state_q)
  FORWARD: begin
    ctr_d = 25'd0;
    if (sr_q[1]) begin
      state_d = BACKUP_LEFT;
    end else if (sl_q[1]) begin
      state_d = BACKUP_RIGHT;
    end
  end
  BACKUP_RIGHT: begin
    if (ctr_q == {26{1'b1}})
      state_d = TURN_RIGHT;
  end
  TURN_RIGHT: begin
    if (ctr_q == {25{1'b1}})
      state_d = FORWARD;
  end
  BACKUP_LEFT: begin
    if (ctr_q == {26{1'b1}})
      state_d = TURN_LEFT;
  end
  TURN_LEFT: begin
    if (ctr_q == {25{1'b1}})
      state_d = FORWARD;
  end
  default: state_d = FORWARD;
endcase
```

```
unique case (state_q)
    FORWARD: begin
      left  = 8'd255;
      right = 8'd255;
      ctr_d = 0; // reset counter in forward mode
      if (sl_q[1]) begin
        state_d = BACKUP_RIGHT;
        ctr_d   = 0;
      end else if (sr_q[1]) begin
        state_d = BACKUP_LEFT;
        ctr_d   = 0;
      end
    end
    BACKUP_RIGHT: begin
      left  = -8'sd255;
      right = -8'sd255;
      if (ctr_q >= COUNTER_MAX) begin
        state_d = TURN_RIGHT;
        ctr_d   = 0;
      end
    end
    TURN_RIGHT: begin
      left  = 8'd255;
      right = -8'sd255;
      if (ctr_q >= COUNTER_MAX) begin
        state_d = FORWARD;
        ctr_d   = 0;
```

```
    case (state_q)
      FORWARD: begin
        left = 8'h50;
        right = 8'h50;
      end
      BACKUP_RIGHT: begin
        left = 8'hB0;
        right = 8'hB0;
      end
      TURN_RIGHT: begin
        left = 8'hB0;
        right = 8'h50;
      end
      BACKUP_LEFT: begin
        left = 8'hB0;
        right = 8'hB0;
      end
      TURN_LEFT: begin
        left = 8'h50;
        right = 8'hB0;
      end
      default: begin
        left = 8'h80;
        right = 8'h80;
      end
    endcase
  end
```

```
    end
    end
    BACKUP_LEFT: begin
      // Reverse
      left  = -8'sd255;
      right = -8'sd255;
      if (ctr_q >= COUNTER_MAX) begin
        state_d = TURN_LEFT;
        ctr_d   = 0;
      end
    end
  TURN_LEFT: begin
      // Rotate left
      left  = -8'sd255;
      right = 8'd255;
      if (ctr_q >= COUNTER_MAX) begin
        state_d = FORWARD;
        ctr_d   = 0;
      end
    end

default: begin
      state_d = FORWARD;
      ctr_d   = 0;
    end
  endcase
end
```

FSM to Control a Robot

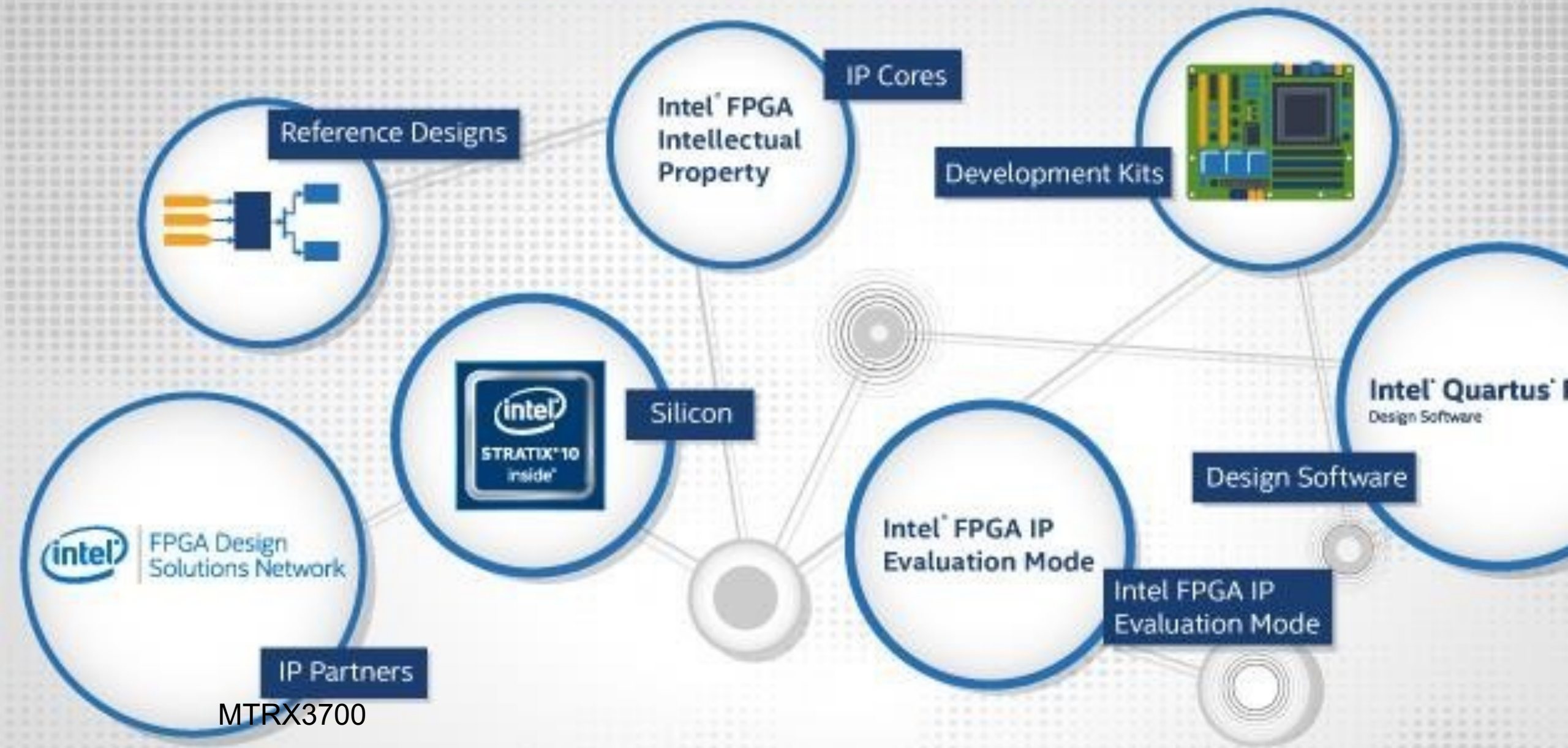
```
  always @(posedge clk) begin
    if (rst) begin
      state_q <= FORWARD;
    end
    else begin
      state_q <= state_d;
    end
    ctr_q <= ctr_d;
    sl_q <= sl_d;
    sr_q <= sr_d;
  end

endmodule
```

```
always_ff @(posedge clk or posedge rst) begin
   if (rst) begin
     state_q <= FORWARD;
     ctr_q   <= 0;
     sl_q    <= 2'b0;
     sr_q    <= 2'b0;
   end
   else begin
     state_q <= state_d;
     ctr_q   <= ctr_d;
     sl_q    <= sl_d;
     sr_q    <= sr_d;
   end
 end

endmodule
```

Intel® FPGA IP Ecosystem



IP Cores

- An **IP (intellectual property) Core** is a block of logic or data that is used in making a field programmable gate array (FPGA) or application-specific integrated circuit for a product.
- **IP Cores** fall into one of three categories: *hard cores* , *firm cores* , or *soft cores* .
  - ***Hard*** *cores* are physical manifestations of the IP design.
  - ***Firm*** (sometimes called *semi-hard*) *cores* also carry placement data but are configurable to various applications.
  - ***Soft*** *cores* exist either as a *netlist* or hardware description language HDL code.
- **IP Cores** includes Processors, DSP, Communications, Interface, Memory, Audio/Video, Controls and Security IP in a range from simple devices, such as counters all the way to complex devices like 32-bit customized soft processors.
- IP Cores can be included in your design in a number of ways:
  - You can **instantiate** the cores in your HDL design using the top-level port definitions.
  - IP can be added as a **block diagram** element in a high-level system diagram with wired connections added graphically.
  - Use a **system level design tool** like Qsys which allows you to incorporate the IP into the design directly.

Intel Altera IP Cores

- **IP Included In Quartus**

  Intel delivers intellectual property (IP) as part of the IP catalogue, which is built into the Intel® Quartus® Prime and Quartus II software.

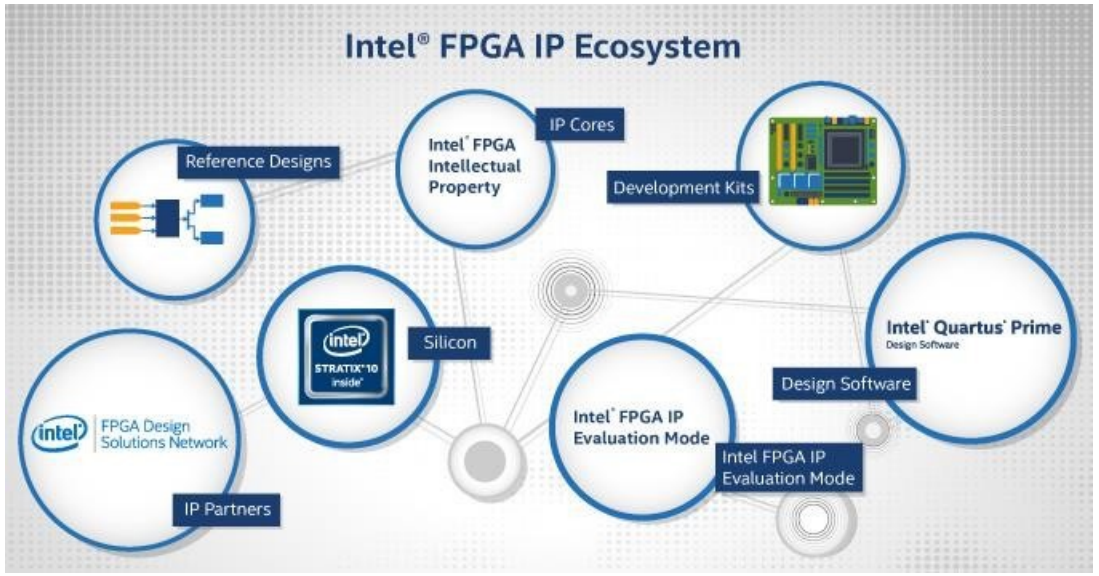
- **Broad Portfolio**

  The portfolio includes IP for protocol and memory interfaces, digital signal processing (DSP), embedded processors, and related peripherals.

- **Tested**

  All the IP cores have been thoroughly tested and verified on hardware to provide you the assurance that they will work well together in your system.

Intel Altera IP Cores



- **Resources -IP Product Portfolio**
- PCI Express*
- Interlaken
- External Memory Interfaces
- JESD204B
- JESD204C
- RapidIO*
- Optical Transport (OTN, FlexE)
- Ethernet
- SerialLite III
- SerialLite IV
- Video Processing
- Connectivity
- Nios® V Processor
- Nios® II Processor
- CPRI

Intel Altera IP Cores

ALTERA_CORDIC	DSP	ROM: 1-PORT	Memory	Clocked Audio Input	Audio/Video	NIOS II Bitswap	Processors
ALTERA_MULT_ADD	DSP	ROM: 2-PORT	Memory	Clocked Audio Output	Audio/Video	NIOS II Floating Point Hardware	Processors
ALTFP_ADD_SUB	DSP	Altera SignalTap II Logic Analyzer	Utility	DisplayPort	Audio/Video	Altera I2C Slave To Avalon MM Master Bridge	Processors
ALFPT_ATAN	DSP	Altera In-System Sources and Probes	Utility	SDI II	Audio/Video	Lauterbach Trace Interface IP	Processors
ALTFP_DIV	DSP	USB Debug Link	Utility	Ethernet 10G MAC	Communications	Vectored Interrupt Controller	Processors
ALTFP_EXP	DSP	Altera Avalon Interrupt Sink	Processor	XAUI PHY	Interface	16x2 Character Display	Audio/Video
ALTFP_LOG	DSP	Altera Avalon Interrupt Source	Processor	TSE	Communications	Audio and Video Config	Audio/Video
ALTFP_MULT	DSP	Altera Avalon-MM Monitor	Interface	Ethernet IEEE 1588 TOD Synchronizer	Communications	IrDA UART	Communications
ALTFP_SINCOS	DSP	Altera Avalon-ST Monitor	Interface	Ethernet IEEE 1588 Time of Day Clock	Communications	RS232 UART	Communications
ALTFP_SQRT	DSP	Altera Clock Source BFM	Utility	Ethernet Packet Classifier	Communications	PS/2 Controller	Interface
LPM_ADD_SUB	DSP	Altera External Memory BFM	Interface	JESD204B	Interface	USB Controller	Communications
LPM_COUNTER	DSP	Altera Avalon Data Pattern Checker	Interface	PHY IP Core for PCI Express (PIPE) v16.1	Interface	Altera UP Flash Memory IP Core	Memory
LPM_DIVIDE	DSP	Altera Avalon Data Pattern Generator	Interface	RapidIO II (IDLE2 up to 6.25 Gbaud)	Interface	SD Card Interface	Memory
LPM_MULT	DSP	Avalon-MM Traffic Generator and BIST Engine	Utility	Avalon-ST Serial Peripheral Interface (SPI)	Interface	SRAM/SSRAM Controller	Memory
JTAG to Avalon Master Bridge	Utility	BCH	Security	SerialLite II v16.1	Interface	100G Ethernet Intel® FPGA IP Core	Communications
ALTCLKCTRL	Utility	High-Speed Reed-Solomon	Security	Transceiver PHY Reset Controller	Interface	1G/2.5G/5G/10G Multi-Rate Ethernet PHY Intel FPGA IP	Communications
Altera PLL	Utility	LDPC	Security	Transceiver Reconfiguration Controller	Interface	10G PDH/SONET/SDH Packet Mapper (TPW192-S)	Communications
Altera Config Debug Agent	Utility	Random Number Generator	Security	Custom PHY v16.1	Interface	40 Gbps Ethernet MAC and PHY	Communications
Altera Remote Update	Utility	Turbo	Security	Cyclone V Transceiver PLL v16.1	Interface	Intel FPGA HDMI IP Core	Audio/Video
Altera Serial Flash Loader	Utility	Viterbi	Communications	Hybrid Memory Cube (HMC) MegaCore	Memory	Intel® FPGA IP for PCI Express*	Communications
Internal Oscillator	Utility	CIC	DSP	DDR2 SDRAM Controller with UniPHY	Memory	Low Latency Ethernet 100G MAC and PHY Intel FPGA IP	Communications
Altera Advanced SEU Detection	Security	FIR II	DSP	DDR3 SDRAM Controller with UniPHY	Memory	PCI Express	Interface
ALTDDIO_BIDIR	Controls	Floating Point Hardware 2 Combinatorial	DSP	LPDDR2 SDRAM Controller with UniPHY	Memory	QDR II SRAM Controller Intel® FPGA IP	Memory
LPM_SHIFTREG	Utility	Floating Point Hardware 2 Multi-cycle	DSP			SerialLite IV Streaming Intel FPGA IP	Communications
FIFO	Memory	NCO	Utility			Video and Image Processing Suite Intel® FPGA IP	Audio/Video
RAM: 1-PORT	Memory	FFT	DSP				
RAM: 2-PORT	Memory						

https://www.intel.com/content/www/us/en/docs/programmable/683102/21-3/introduction-to-cores.html

small robot

Search this document

- 1. Introduction to Intel® FPGA IP Cores
  - 1.1. IP Catalog and Parameter Editor
  - 1.2. Installing and Licensing Intel® FPGA IP Cores
  - 1.3. Best Practices for Intel® FPGA IP
  - 1.4. IP General Settings
  - 1.5. Specifying the IP Core Parameters and Options (Intel® Quartus® Prime Pro Edition)
  - 1.6. Generating IP Cores (Intel® Quartus® Prime Standard Edition)
  - 1.7. Modifying an IP Variation
  - 1.8. Upgrading IP Cores
  - 1.9. Simulating Intel® FPGA IP Cores
  - 1.10. Synthesizing IP Cores in Other EDA Tools
  - 1.11. Support for the IEEE 1735 Encryption Standard
  - 1.12. Introduction to Intel® FPGA IP Cores Archives
  - 1.13. Introduction to Intel® FPGA IP Cores Revision History

1. Introduction to Intel® FPGA IP Cores

Updated for:

Intel® Quartus® Prime Design Suite **21.3**

Intel and strategic IP partners offer a broad portfolio of configurable IP cores optimized for Intel FPGA devices.
The Intel® Quartus® Prime software installation includes the Intel® FPGA IP library. Integrate optimized and verified Intel® FPGA IP cores into your design to shorten design cycles and maximize performance. The Intel® Quartus® Prime software also supports integration of IP cores from other sources. Use the IP Catalog (**Tools > IP Catalog**) to efficiently parameterize and generate synthesis and simulation files for your custom IP variation. The Intel® FPGA IP library includes the following types of IP cores:

Basic functions	Interface protocols
Bridges and adapters	Low power functions
DSP functions	Memory interfaces and controllers
Intel FPGA interconnect	Processors and peripherals

This document provides basic information about parameterizing, generating, upgrading, and simulating stand-alone IP cores in the Intel® Quartus® Prime software.

Figure 1. Intel® FPGA IP Catalog



Intel® FPGA Intellectual Property

The Intel® FPGA Intellectual Property (IP) portfolio covers a wide variety of applications with their combination of soft and hardened IP cores along with reference designs. Our IP partners (via Intel Partner Alliance) contribute to this portfolio and provide alternative solutions.

Self-Service Licensing Center

An online licensing center that allows you to view, obtain, activate, and manage your Intel® FPGA software licenses.

Intel® FPGA Partner Intellectual Property

Browse catalog of Intel® FPGA partner intellectual property in the Intel® Solutions Marketplace.

IP Evaluation and Purchase

Evaluation mode and purchasing information for Intel® FPGA Intellectual Property.

Search this collection

12 results

Filter By

CLEAR ALL FILTERS

Intel × Cyclone® IV FPGAs ×

IP Category

- Basic Functions (2)
- DSP (7)
- Interface Protocols (3)
- Memory Interfaces and Controll... (1)

Product Name	Provider	Supported Devices	Platform Designer Compliant
CIC Intel® FPGA IP Core	Intel	Stratix® IV GX FPGA; Cyclone® V GX FPGA;	

Resources

https://www.fpga4student.com/p/verilog-project.html

https://verilogguide.readthedocs.io/en/latest/index.html

https://hdlbits.01xz.net/wiki

https://nandland.com/learn-verilog

https://eda-playground.readthedocs.io/en/latest/code-examples/verilog.html